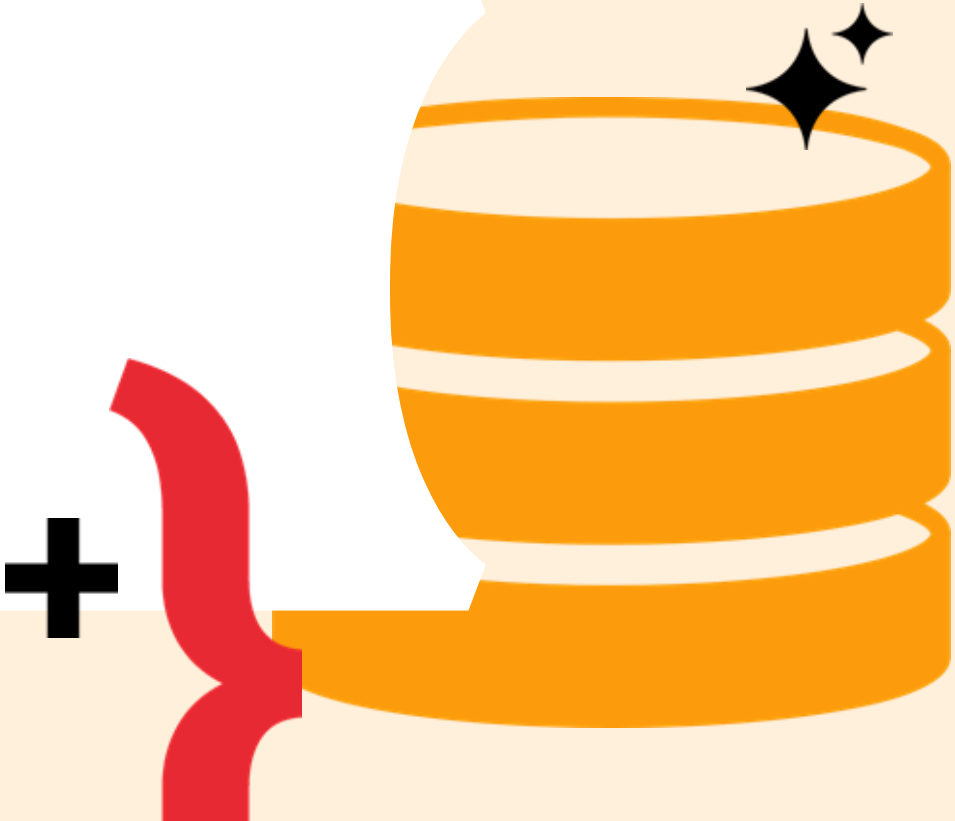
Couchbase SQL++ Fundamentals



March 2026

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ | Innovate, Iterate, and Scale apps faster

SQL-like queries for JSON

- Flexible data schema
- ANSI standard, including JOINS
- User Defined Functions
- Distributed SQL transactions

Query without compromise

- High-performance query engine
- Multidimensional scaling (MDS)
- Cost-based query optimizer
- Rich Indexing Options

Familiar tooling

- Create and test queries in the Query Workbench
- Visual Query Plan
- Index Advisor to optimize queries

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ | Familiar, Optimized and Simple

MongoDB Query Language

```
db.activity.aggregate(
 { $match: { startDate: { $gt: "2018-01-01",
                          $lt: "2018-12-31" } } },
 {$lookup: {
        from: "account",
        localField: "accid",
        foreignField: "id",
        as: "account_docs"
          }
 },
 { $match: { "account_docs": { $ne: [] } } },
 { $unwind: "$account_docs" },
 {
   $project: {
     item: 1,
     task: { $cond:{if:
      { $eq: ["$activityType", "Task"] }, then: 1,
          else: 0 } },
     appt: { $cond: { if:
      { $eq: ["$activityType", "Appointment"] },
then: 1,
          else: 0 } } }
  },
  {
     $group: {
          _id: "$account_docs.industry",
          tasks: { $sum: "$task" },
          appointments: { $sum: "$appt" }
  } }
);
```

Relational SQL

```
SELECT ac.industry,
 SUM(CASE WHEN a.activitytype = 'Task'
    THEN 1 ELSE 0 END ) task,
 SUM(CASE WHEN a.activitytype ='Appointment'
    THEN 1 ELSE 0 END ) appts
FROM crm.activity a
  INNER JOIN crm.account ac
      ON (a.accid = ac.id)
WHERE a.startdate BETWEEN '2018-10-01'
           AND   '2018-12-31'
GROUP BY ac.industry
```

Couchbase SQL++

```
SELECT ac.industry,
 SUM(CASE WHEN a.activitytype = 'Task'
    THEN 1 ELSE 0 END ) task,
 SUM(CASE WHEN a.activitytype ='Appointment'
    THEN 1 ELSE 0 END ) appts
FROM crm.activity a
  INNER JOIN crm.account ac
      ON (a.accid = ac.id)
WHERE a.startdate BETWEEN '2018-10-01'
           AND   '2018-12-31' `
GROUP BY ac.industry
```

- ✓ Couchbase SQL++ is *similar* to Relational SQL, hence **familiar to developers**
- ✓ Couchbase SQL++ *declarative*, allowing **automatic optimizations**
- ✓ Couchbase SQL++ provides natively integrated *JOINS*, to **simplify queries**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ | Combining JOINS with UNNEST & ARRAY predicates

Query: List all customers and their contacts who might be at the meetings that I will be attending next month

ORACLE SQL

```
SELECT  a.id,  a.title meeting,
  c.name customer,   a.startdate,  cn.name contact_name,
cn.title contact_title,  cn.email contact_email
FROM  activity a
  INNER JOIN account c ON a.accid = c.id
  INNER JOIN contact cn ON c.id = cn.accid
  INNER JOIN activityparticipant p ON a.id = p.actid
WHERE a.activitytype = 'Appointment'
  AND a.startdate BETWEEN '2024-01-01' AND '2024-01-31'
  AND p.userid = ‘usr73’
```

- *activity, accounts, contacts, and activityparticipants* are separate **normalized** tables
- This SQL query requires **3 JOINS** across all tables

COUCHBASE SQL++

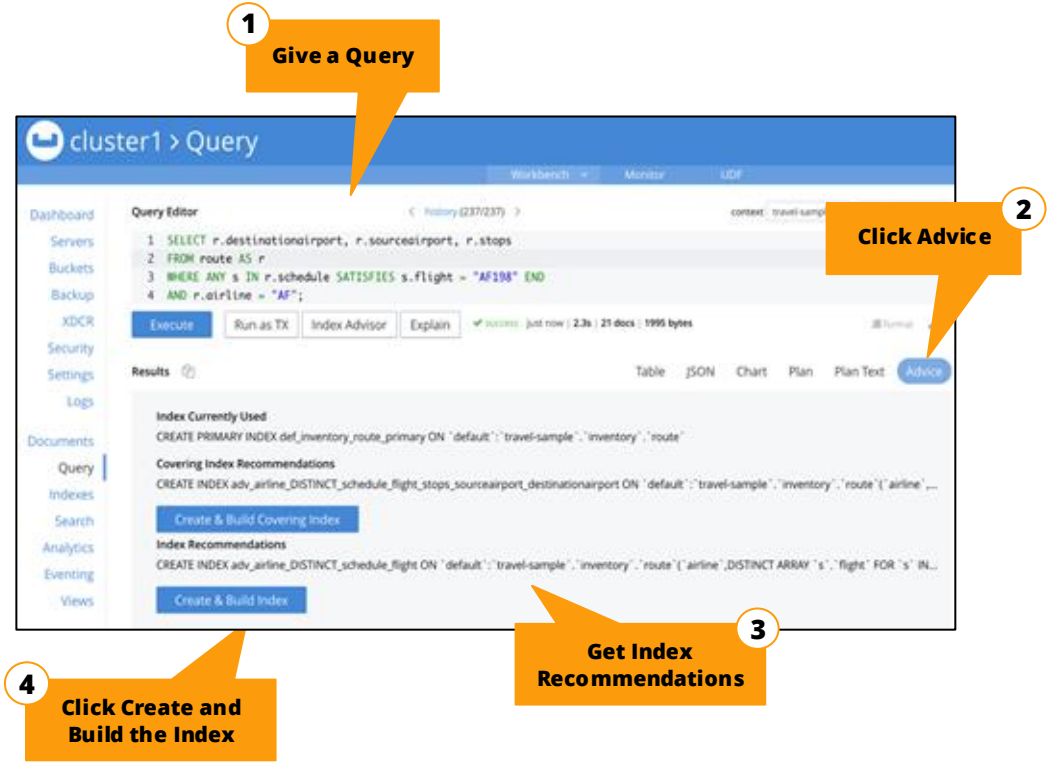
```
SELECT  a.id,  a.title meeting,
  c.name customer,  a.startDate,  cn.name contact_name,
  cn.title contact_title,  cn.email contact_email
FROM activity a
  INNER JOIN account c ON a.accid = c.id
  UNNEST c.contacts AS cn
WHERE a.activityType = 'Appointment'
  AND a.startDate BETWEEN '2024-01-01' AND '2024-01-31'
  AND ANY p IN a.participants SATISFIES p.userid = 'usr73'
```

- *contacts* are nested into *accounts* documents, and *participants* are nested into *activity* documents *as* **denormalized** JSON arrays
- This SQL++ query uses **UNNEST** to access nested *contacts* within *accounts;* and **ANY…IN** to filter *activity* based on nested *participants*

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ | Index Advisor

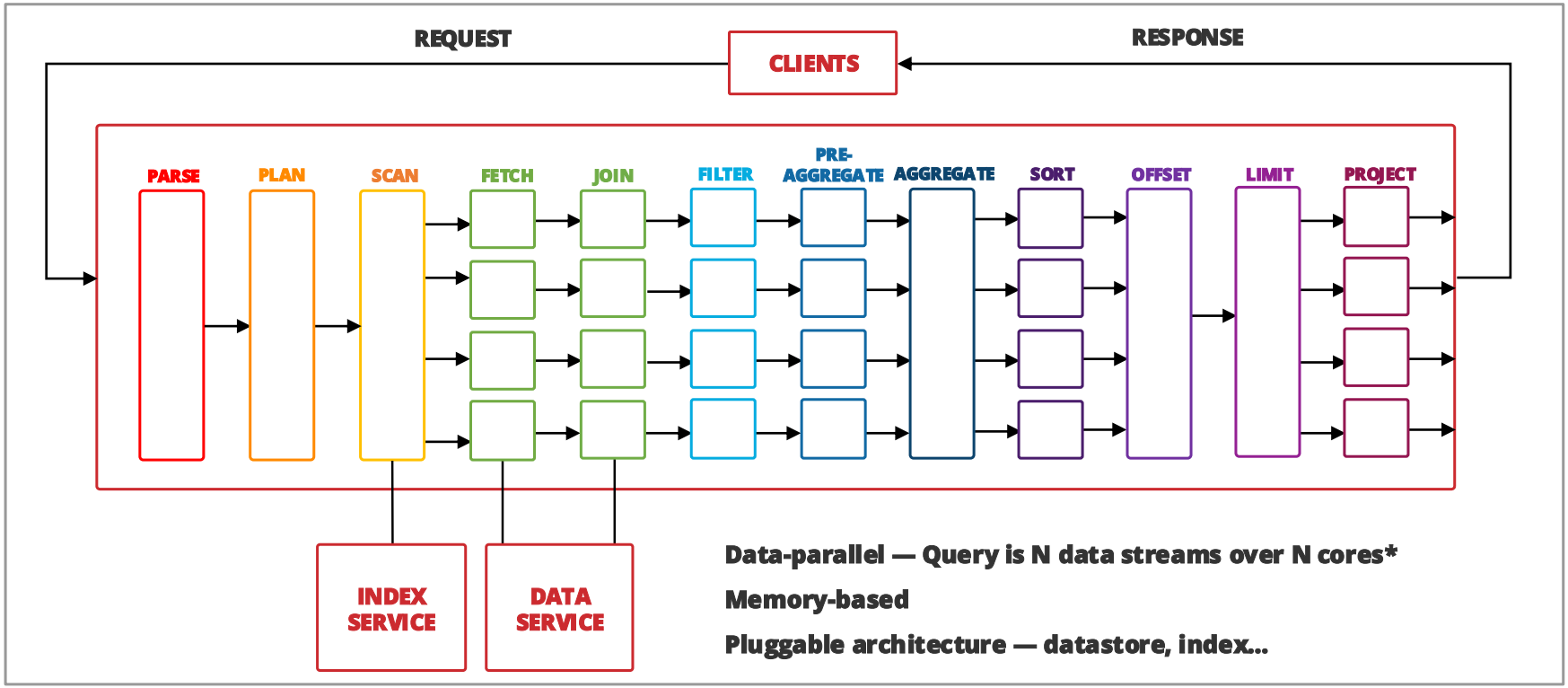
- With the Index Advisor, Couchbase will suggest the appropriate indexes **to speed up a given query**, taking the guesswork out of query tuning
- The Index Advisor can also **monitor and analyze the statistics** collected from running a workload, and suggest the indexes that will speed up the queries in the workload
- Index Advisor greatly **reduces the complexity** and efforts required for enterprise developers and operations engineers to determine the right indexes to speed up their queries



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

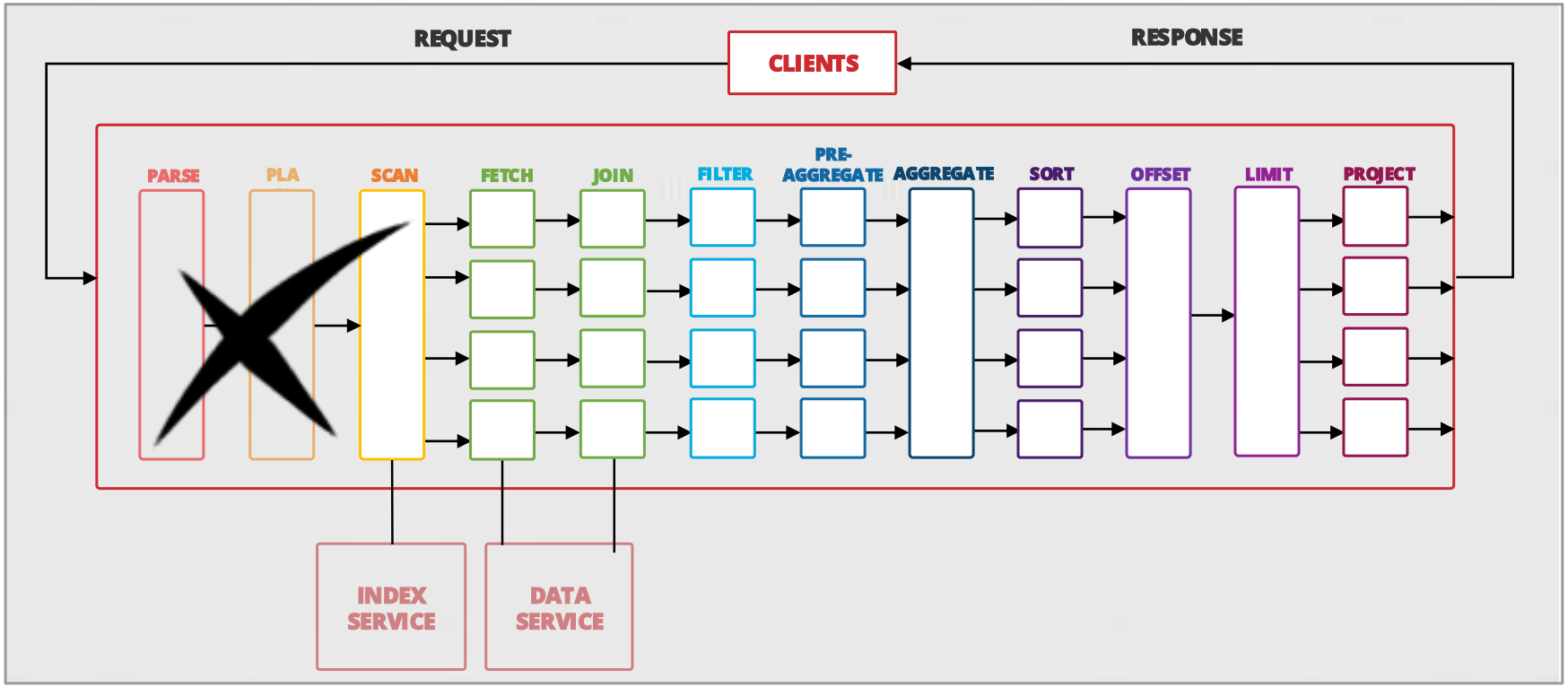
SQL++ Query

SQL++ Query: Full Pipeline



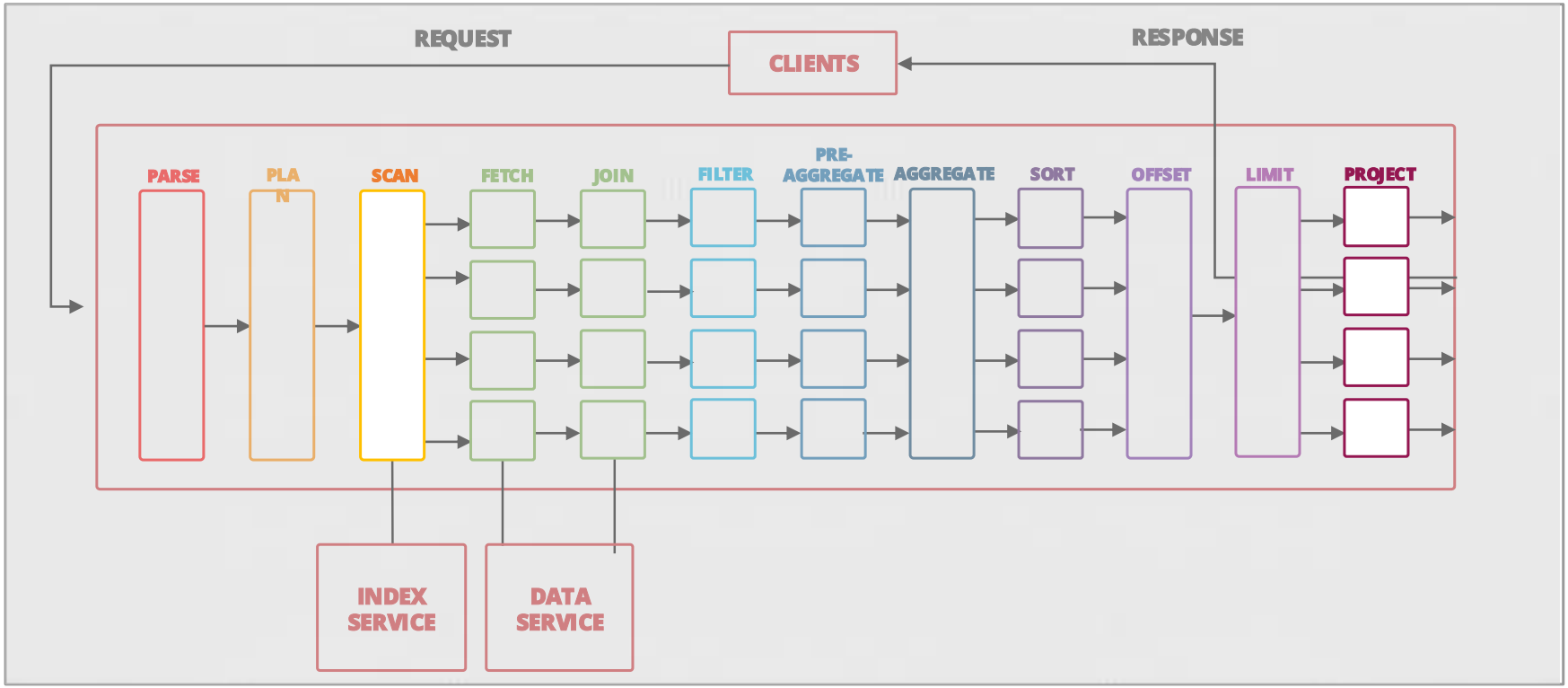
Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Query: PREPARED Statements



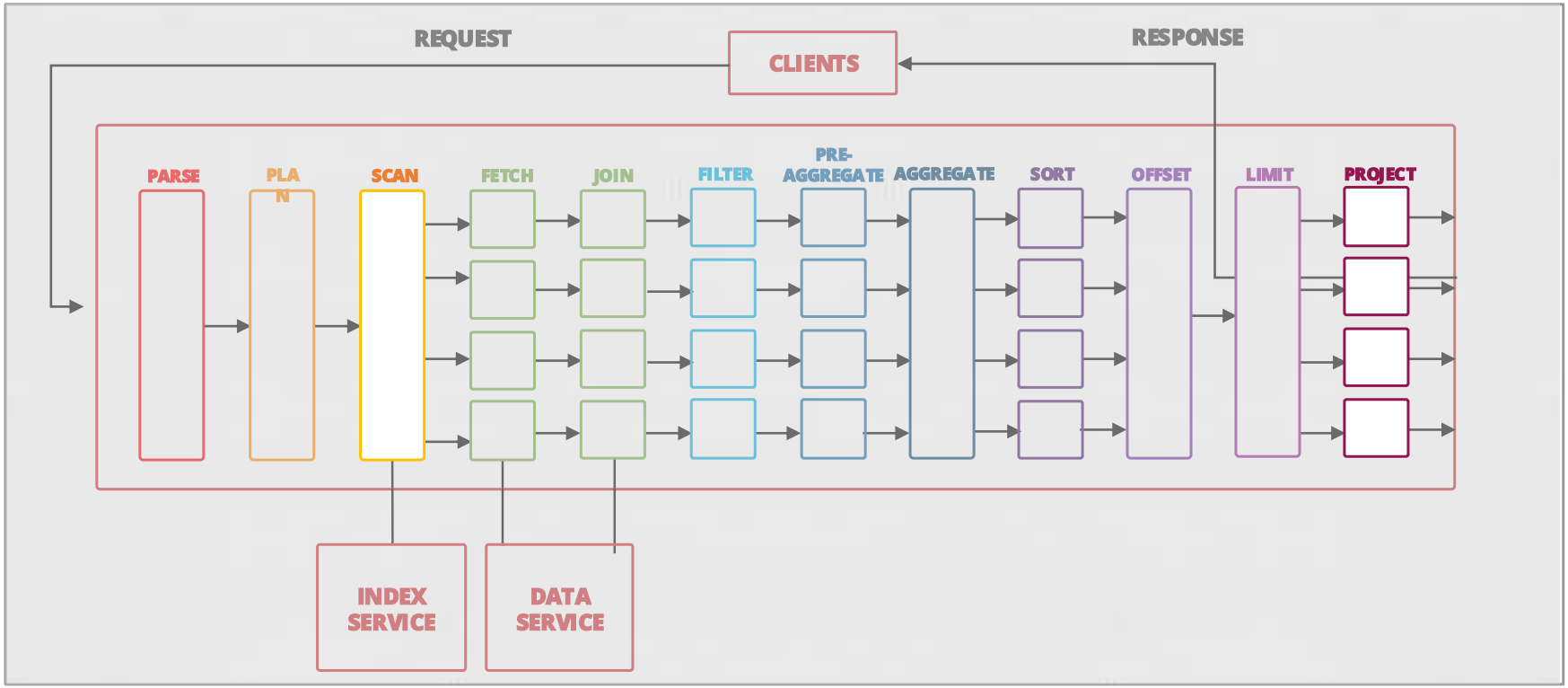
Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Query: PREPARED Statements + Covering Index



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Query: PREPARED Statements + Covering Index



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Query: Two Approaches

There are two approaches to implementing prepared statements, choose one that best fits your environment

- **SDK Prepareds –** Prepareds are managed by the SDK internally - To indicate that an SDK should optimize a query, the adhoc parameter should be set to false.

```
QueryResult result = cluster.query(
    "select count(*) from `travel-sample`.inventory.airport where country = ?",
    QueryOptions.queryOptions()
        .adhoc(false)
        .parameters(JsonArray.from("France"))
);
```

- **Named Prepareds –** This method of prepared statements is similar to the previous option but instead of the SDK managing the planning of the queries, it would be managed by the application or through an external process by first preparing the statement

```
PREPARE unique_name_for_query FROM
SELECT airportname FROM `travel-sample` WHERE country=$1
```

Once the query has been prepared, the query plan is cached in the Query service and can be executed by using an EXECUTE Statement.

```
EXECUTE unique_name_for_query
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Query: A few pointers

Use Array to Eliminate OR statements

Discontinue the use of USE INDEX Statements

Use Parameterized Queries

Discontinue the use of SELECT *

Use Common Table Expressions (CTE) to optimize JOINS

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Examples | User Defined Function with Javascript

UDF to query an organizational **hierarchy**

USE CASE 1

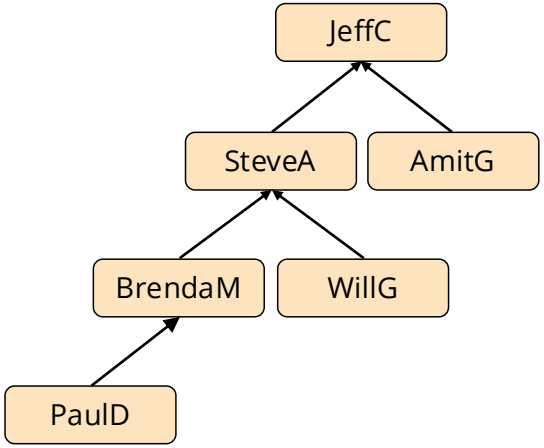
UDF to perform a **graph** traversal

USE CASE 2

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Use Case 1 | UDF to query organizational hierarchy

Organizational hierarchy



JSON documents

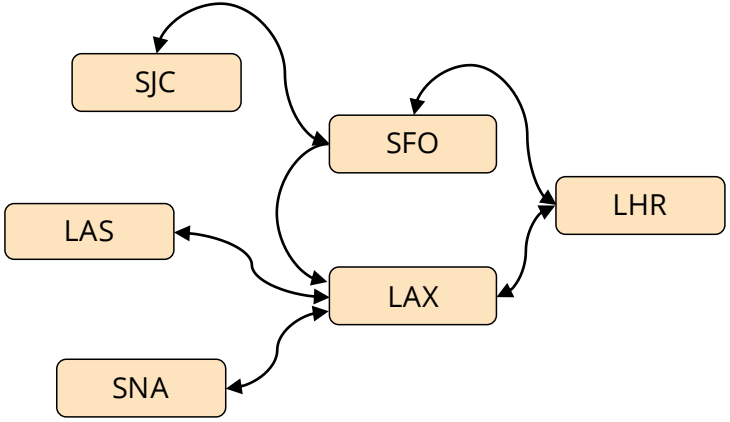
```
[
  {"empid":1, "name": "JeffC"},
  {"empid":2, "name": "SteveA",  "reportsTo": "JeffC"},
  {"empid":3, "name": "AmitG",   "reportsTo": "JeffC"},
  {"empid":4, "name": "BrendaM", "reportsTo": "SteveA"},
  {"empid":5, "name": "WillG",   "reportsTo": "SteveA"},
  {"empid":6, "name": "PaulD",   "reportsTo": "BrendaM"}
]
```

How do you query this hierarchy ***without using*** JOIN
so the number of levels you can support is not fixed?

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Use Case 2 | UDF to perform a graph traversal

Graph traversal



JSON documents

```
{
    "airline": "AA",
    "airline_id": 24,
    "codeshare": "",
    "dest_airport": "LAX",
    "dest_airport_id": 3484,
    "equipment": "77W",
    "source_airport": "LHR",
    "source_airport_id": 507,
    "stops": 0
}
```

```
{
    "airline": "AA",
    "airline_id": 24,
    "codeshare": "Y",
    "dest_airport": "SFO",
    "dest_airport_id": 3469,
    "equipment": 744,
    "source_airport": "LHR",
    "source_airport_id": 507,
    "stops": 0
}
```

If I start from an airport, how do I get to another one?
How can I choose the number of maximum hops?

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

UDF with Javascript and SQL++

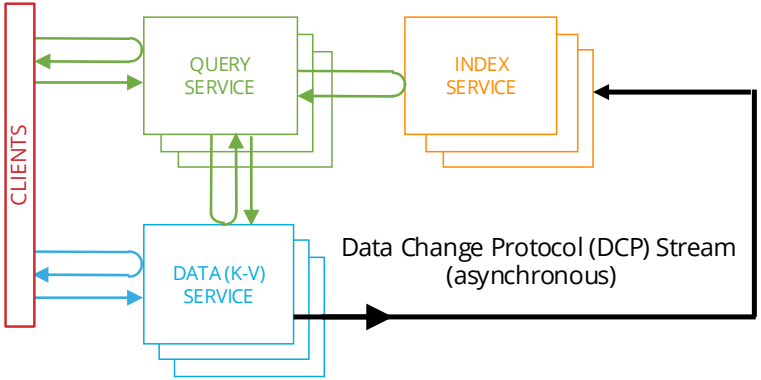
You can **perform complex queries** such as graph traversals.

You can easily **adapt public Javascript** code to Couchbase.

Test it out! The code and full example is described in this blog.
https://www.couchbase.com/blog/traverse-hierarchy-user-defined-functions-in-7-1/

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Read your own writes



Because the indices are updated asynchronously, it is possible for the index to be slightly out of date (behind) wrt the data. If you need to immediately query your own writes, and expect your recent mutations to be included in the query result ...

```
queryOptions().scanConsistency(QueryScanConsistency.REQUEST_PLUS)
```

This delays the query until the indices are up to date to the time the request is made.

```
queryOptions().consistentWith(mutationState)
```

This delays the query until the indices include all changes in the mutation state. This option will tend to be faster than REQUEST_PLUS

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Mutation Tokens

When enabled in a client, all CRUD operations can return Mutation Tokens.

```
CouchbaseEnvironment env = DefaultCouchbaseEnvironment
.builder()
   .mutationTokensEnabled(true)
```

The document returned from a k-v operation contains additional information, including Mutation Tokens

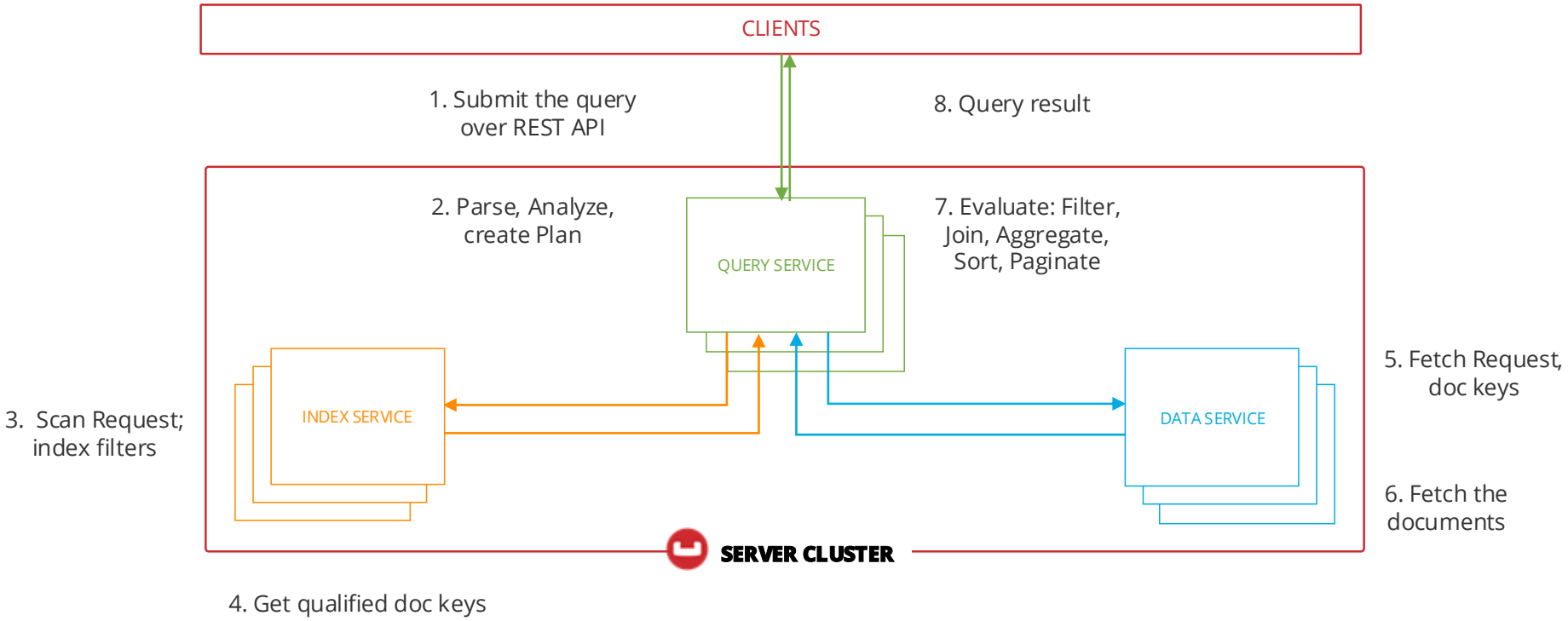
```
result1 = collection.upsert(key1, doc1);
result2 = collection.upsert(key2, doc2);
mutationState = MutationState.from(result1, result2);
```

This MutationState can be used when performing queries in order to meet the data consistency needs of the client.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

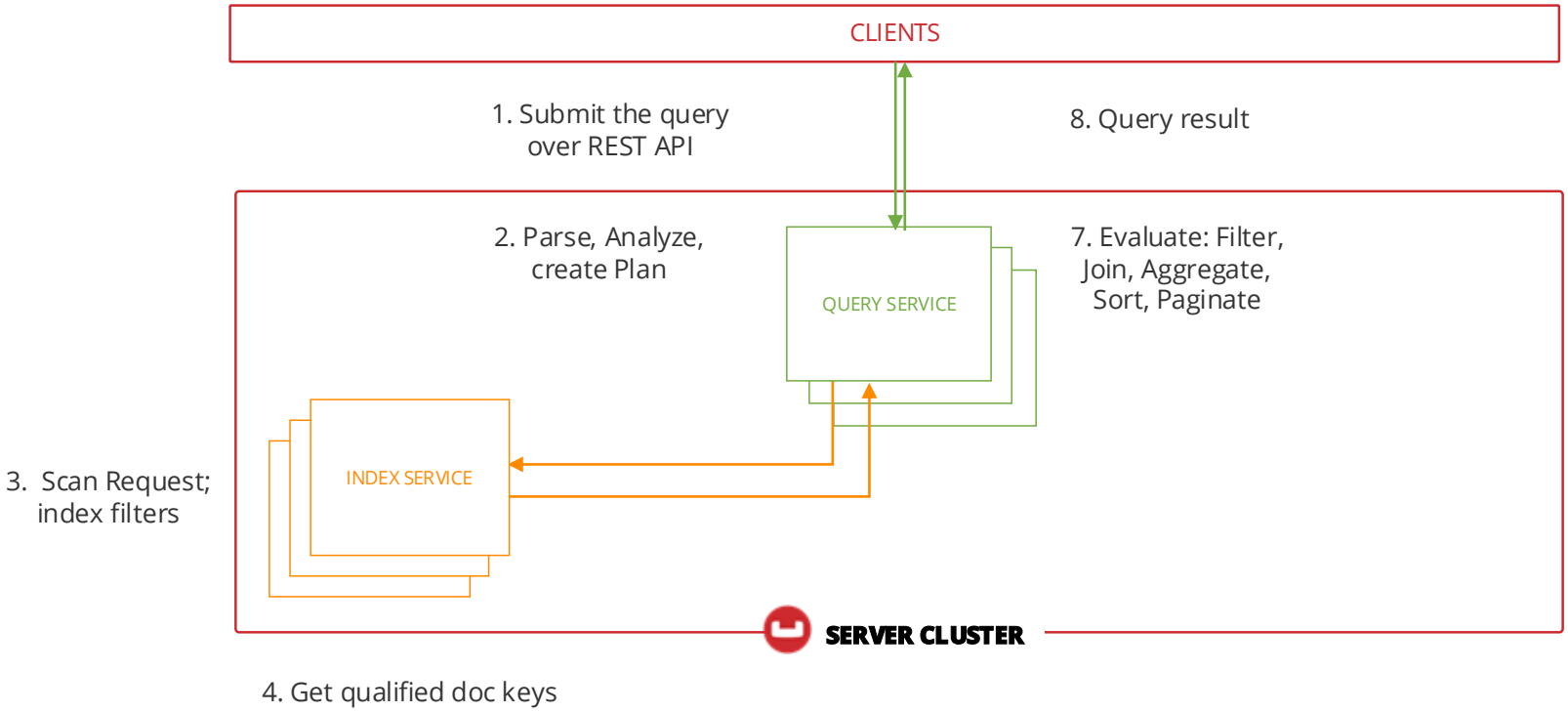
SQL++ Indexes

Index Usage



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Improved query path with Covered Index



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Indexes: Example – Equality Predicates do not need to be Indexed

Severity: High

- **Description:** It is unnecessary and detrimental to performance to index values which are equality predicates within the index. Consider the following example, the query is for values of status that are Pending , this is useful as an index filter, but including status in the index key causes all values of “Pending” to be indexed which yields no performance benefits and increases the index scan size.

```
SELECT orderId, orderDate, orderTotal
FROM ecommerce
WHERE docType = 'Order'
 AND orderDate = '2021-03-15'
 AND status = ‘Pending’
```

Bad Example:

```
CREATE INDEX idx_pending_orders ON ecommerce (
orderDate, status )
WHERE docType = 'Order' AND status = 'Pending'
```

Correct Example:

```
CREATE INDEX idx_pending_orders ON ecommerce (
orderDate )
WHERE docType = 'Order' AND status = 'Pending'
```

- **Suggested Next Steps:** Update any indexes to remove fields which are also an equality predicate within the index

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

SQL++ Indexes: Example – Indexing Functional Expressions

Severity: High

- When performing a function on an attribute such as UPPER() the function should be present in the index to avoid a full index scan. This will reduce latency by preventing the function being applied real-time to every value in the index, instead of only once at indexing time.

Query:

```
SELECT userId, username, email, address
FROM ecommerce
WHERE docType = "User"
    AND UPPER(firstName) = UPPER('John')
```

Incorrect Index:

```
CREATE INDEX `idx_firstName` ON `bucket` (
  firstName
)
WHERE `docType` = "User"
```

Correct Index:

```
CREATE INDEX `idx_upper_firstName` ON `bucket` (
  UPPER(firstName)
)
WHERE `docType` = "User"
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2025. All rights reserved.

Thank you!

Couchbase